

Hierarchical Facility Location Problem – Report

Daniele Bugatti & Giovanni Gaio

June 10, 2026

1 Introduction

Efficient distribution of medical facilities is crucial for maximizing territorial accessibility and minimizing healthcare response times. Historically, public health infrastructure has often developed via uncoordinated local initiatives, resulting in regional inefficiencies and hospital oversupply. To address this, we propose a centralized planning strategy that utilizes population distribution to minimize the average travel distance to the nearest healthcare provider.

We incorporate facility differentiation by modeling a two-level nested hierarchy: Level-1 local clinics provide widespread baseline coverage, while Level-2 hospital hubs are highly resource-intensive, fewer in number, and encompass all services, including those of Level-1 sites.

The challenge is modeled as a graph optimization problem over a vertex set V . Given a target number of facilities k_1 and k_2 ($k_2 \leq k_1$), we seek to select two nested subsets of nodes, S_1 and S_2 , subject to:

$$\begin{cases} |S_1| = k_1 \\ |S_2| = k_2 \\ S_2 \subseteq S_1 \end{cases}$$

The hierarchical nesting constraint ($S_2 \subseteq S_1$) makes this problem a direct extension of the classic Facility Location Problem. The objective function to minimize represents the total demand-weighted distance across both facility tiers:

$$\sum_{i \in V} h_i \min_{j \in S_1} d_{ij} + \sum_{i \in V} h_i \min_{j \in S_2} d_{ij}$$

where h_i is the population at node i and d_{ij} is the shortest-path distance from node i to node j . Since the combinatorial search space grows exponentially with $|V|$, exact approaches become computationally intractable for large-scale instances. We therefore investigate two approximated strategies, Simulated Annealing (SA) and a Genetic Algorithm (GA), and eval-

uate their performance against an exact Integer Linear Programming (ILP) solver.

2 Methods

Both developed metaheuristics explore the HFLP search space via non-deterministic updates designed to progressively lower, on average, the objective value of the candidate solution, trading theoretical optimality guarantees for computational efficiency.

2.1 Simulated Annealing

We implemented a Simulated Annealing algorithm where, for each move, a single facility chosen at random from both S_1 and S_2 and it placed in another empty node. Of course, if the facility extracted has Level 2, its new location will be chosen among $S_1 \setminus S_2$.

Then, the acceptance criterion follows the standard

$$P(S \rightarrow S') = \begin{cases} 1 & \text{if } \Delta Z < 0 \\ e^{-\frac{\Delta Z}{T}} & \text{if } \Delta Z \geq 0 \end{cases}$$

where Δ is the difference of the objective function in the new and the old configuration.

The temperature follows the bounded geometric progression $T_{k+1} = \min(\alpha T_k, T_{\text{MIN}})$. In our algorithm we fixed $\alpha = 0.995$ and $T_{\text{MIN}} = 10^{-3}$.

2.2 Genetic Algorithm

The GA maintains a population of candidate tuples (S_1, S_2) scored by their total demand-weighted distance. Successive generations are evolved through three sequential operators:

- **Selection:** Parent pairs are chosen using a k -way tournament selection (size $k = 3$) based on individual fitness values.
- **Crossover:** With a probability $p_c = 0.8$, a child's clinic layer S_1 is generated by sampling p_1 nodes from the union of the parents' S_1 sets. The hub layer S_2 is sampled

from the intersection of the child's S_1 and the parents' S_2 union; any remaining capacity is filled directly from S_1 to guarantee feasibility.

- **Mutation:** With a mutation rate $\mu = 0.3$, the offspring undergoes independent random facility swaps within S_1 or S_2 using the same logic as the SA neighborhood operators.
- **Elitism:** the absolute best individual is brought unchanged to the next generation.

3 Results

3.1 Synthetic Data

As a preliminary step we tested on two families of synthetic graphs: random graphs (Erdos-Reyni) and grid graphs. We tested first small graphs, with 50 nodes, then larger graphs with up to 400 nodes.

For the cases where it was possible we compared the solutions from SA and GA with the optimal given by the ILP. For smaller graphs, up to 100 nodes, we observed that SA always found the optimal solution with runtime similar to ILP. GA also achieved an optimal cost in most cases, though with longer runtime.

On larger graphs, where we couldn't evaluate the optimal ILP solution in reasonable times, we observed similar optimized cost for SA and GA. It is notable that SA found consistently better solutions than GA on a roughly equivalent runtime.

3.2 Simulation on the Veneto Region

The model was applied to real-world regional data describing the Veneto Region. We constructed a graph where each node represented one of the municipalities within the region. The distance matrix was computed by determining the geographic centroid of each municipality and evaluating the shortest-path distances between them over the actual road network. Population data and municipal boundaries were derived from the 2021 ISTAT census.

The choice of parameters $k_1 = 28$ and $k_2 = 8$ naturally follows the regional "Hub and Spoke" healthcare model established by the *Deliberazione della Giunta Regionale n. 614 (14 May 2019)*. This institutional act classifies Veneto's

hospitals into 8 high-specialty centers (hubs) and 20 lower-level facilities (spokes).

With this parameters we observed the results shown in Table 1 and Figure 1. Here once again SA comes within 1.4% of the exact optimal cost half of the times, while GA is only within 2.8%. Once again SA is shown to produce consistently higher quality solutions in a lower runtime than GA.

Algorithm	Objective	Relative gap	Runtime (s)
ILP	$3.49 \cdot 10^{10}$	0 %	63.0
GA	$3.59 \cdot 10^{10}$	2.8 %	44.7
SA	$3.54 \cdot 10^{10}$	1.4 %	40.3

Table 1: Short summary of runs on the Veneto Region graph with $k_1 = 28$ and $k_2 = 8$. Values are median.

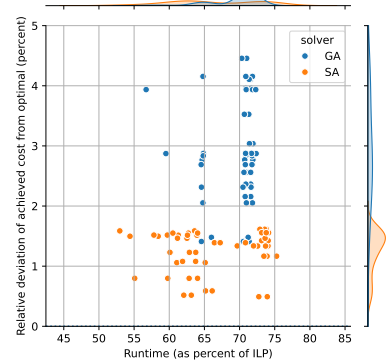


Figure 1: Scatterplot of runtimes and achieved costs on the Veneto Region instance.